

Lesson 4 Goods Sorting

1. Program Logic

In the past, the tasks of packaging and sorting goods were assigned to human, which is low-efficient and error-prone. But today, with vision module, robot or robotic arm can sort the goods by distinguishing the colors and labels of goods helping us complete the task better.

Robot or robotic arm can realize label recognition through locating, image segmentation and contour searching. They will code and decode based on the recognized label.

The program logic of color tracking is the same as the content in “Lesson 2 Color Tracking”, that is, robot and robotic arm use Lab color space to realize color recognition.

Convert the RGB color space to Lab, image binarization, and then perform operations such as expansion and corrosion to obtain an outline containing only the target color. Use circles to frame the color outline to realize object color recognition.

After recognition, robot and robotic arm begin picking goods. They will pick the goods with label first, then pick the goods in terms of color. Each situation has corresponding order criterion.

Goods with labels: they will pick the goods following the order of 1, 2, 3. (If you don't label the goods, this step will be skipped.)

Goods in different colors: after obtaining the recognition area of the corresponding colors, ArmPi FPV will compare the data it gain. It will make multiple judgments and takes the average to compare their sizes. Then it determines the picking order of the goods in the same color based on their

area. Priority will be given to object with larger area.

Having picked, it will sort the goods in terms of label and color, then it will return to original posture.

The source code is located in
/home/ubuntu/armpi_fpv/src/object_sorting/scripts.

```
86 # turn off rgb
87 def turn_off_rgb():
88     led = Led()
89     led.index = 0
90     led.rgb.r = 0
91     led.rgb.g = 0
92     led.rgb.b = 0
93     rgb_pub.publish(led)
94     led.index = 1
95     rgb_pub.publish(led)
96
97 x_dis = 500
98 Y_DIS = 0
99 y_dis = Y_DIS
100 last_x_dis = x_dis
101 last_y_dis = y_dis
102 x_pid = PID.PID(P=0.01, I=0.001, D=0) #pid initialization
103 y_pid = PID.PID(P=0.00001, I=0, D=0)
104
105 tag_x_dis = 500
106 tag_y_dis = 0
107 tag_x_pid = PID.PID(P=0.01, I=0.001, D=0) #pi initialization
108 tag_y_pid = PID.PID(P=0.02, I=0, D=0)
109
110 stop_state = 0
111 move_state = 1
```

2. Operation Steps

i The entered command must be strictly distinguished between uppercase and lowercase and spaces, and the keywords support the "Tab" key completing.

2.1 Preparation

① Please place the map on even desk and the robotic arm should be placed on the corresponding area on the map.

② Prepare 3 blocks in red, green and yellow and 3 blocks labeled with

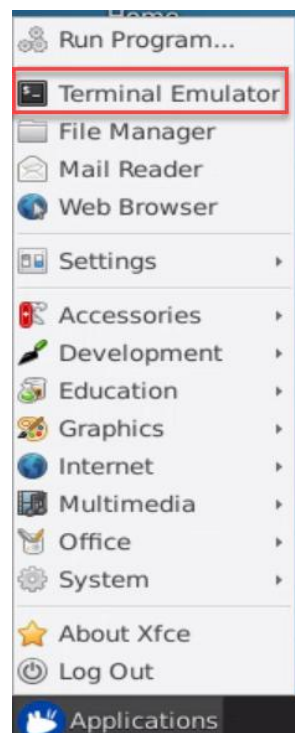
tag1, tag2 and tag3. And place them randomly on the recognition area of the map. Note: the interval between each blocks is not less than 3cm.

- ③ Turn on the robotic arm and wait to finish.

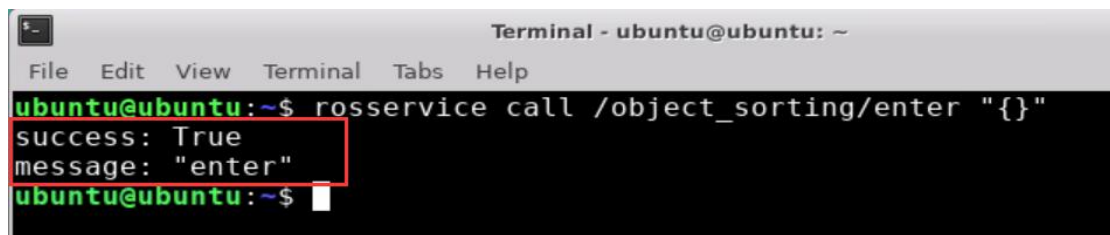
2.2 Enter the game

① Turn on ArmPi FPV and connect to Raspberry Pi desktop through NoMachine.

② Click “Applications” button and select “Terminal Emulator” in pop-up interface to open the command line terminal.



③ Enter “`rosservice call /object_tracking/enter "{}"`” command, then press “Enter” to start the game. After the game starts, a print prompt will appear, as shown in the figure below.

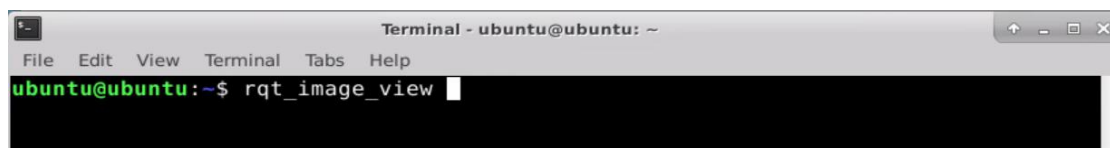


```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosservice call /object_sorting/enter "{}"
success: True
message: "enter"
ubuntu@ubuntu:~$
```

2.3 Turn On the Image

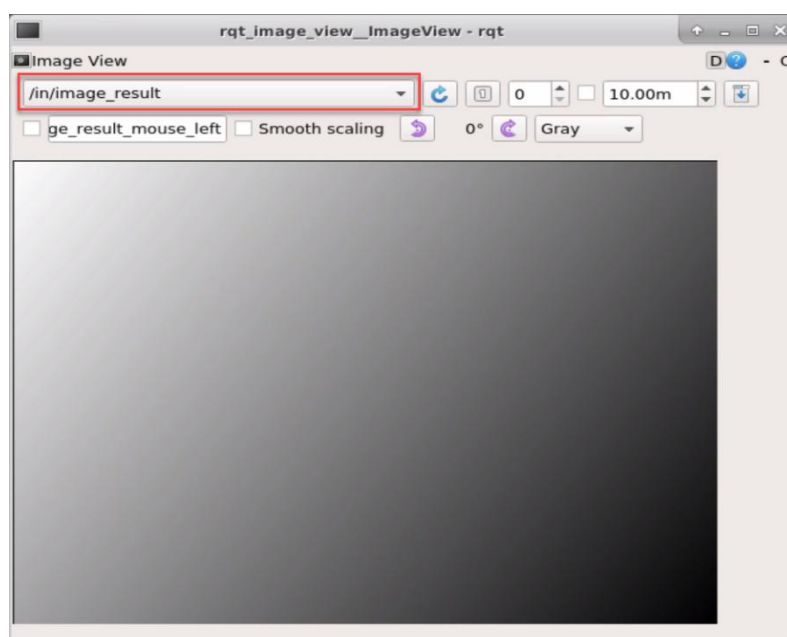
① When entering the game, we need to turn on image returned by camera with rqt. Please go back to the desktop and click the “Application” icon at bottom-left corner to open a new terminal.

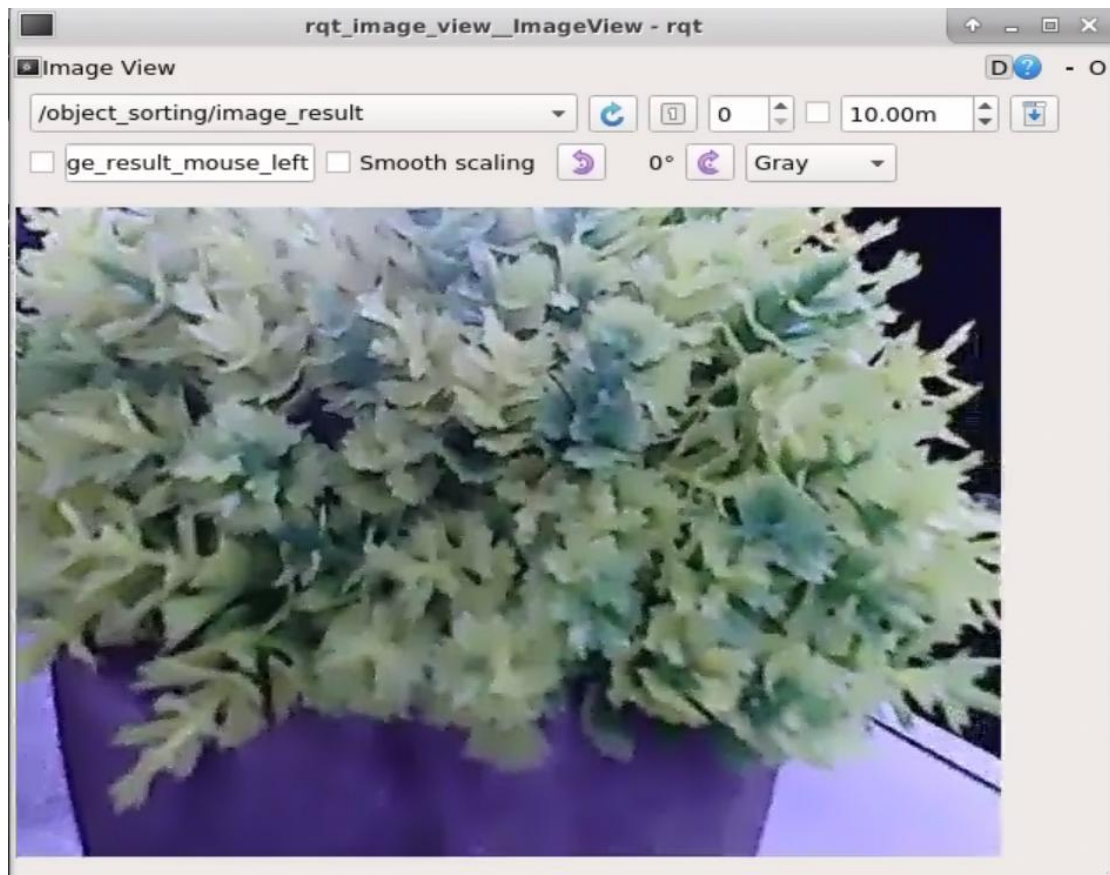
② Enter “rqt_image_view” command and press “Enter”. Then open rqt after few seconds.



```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rqt_image_view
```

③ As shown in the following figure, select “/in/image_result” and other settings remain the same.





Note: after the image is turned on, please be sure to select the topic corresponding to the game you play. Otherwise, the recognition process can not be displayed normally if you start other games.

2.4 Start the Game

① Please go back to the terminal opened in “2.1 Enter the Game” and enter “rosservice call /object_sorting/set_running "data: true"” command.

When the prompt, shown in the red box, appears, it means you start the game successfully.

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_running "data: true"
success: True
message: "set_running"
ubuntu@ubuntu:~$
```

② After starting the game, we need to select the targeted color and labels. If we need to pick red blocks and blocks with tag1 label, we need to enter “rosservice call /object_sorting/set_target "color:

- 'red'

tag:

- 'tag1' "" .

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_target "color:
- 'red'
tag:
- 'tag1' "
success: True
message: "set_target"
ubuntu@ubuntu:~$
```

③ If we need to sort multiple goods, we should enter several parameters. If we want to sort 6 goods in terms of red, green, blue, tag1, tag2 and tag3, the input format should be dash + space + ‘color/label’. Please follow this format strictly. The operation steps are as follow.

① Please enter “rosservice call /object_sorting/set_target” command and press “Tab” to complete.

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_target "color:
- 'red'
tag:
- 'tag1' ""
ubuntu@ubuntu:~$
```

② Enter “red” parameter according to the previous steps, then press “Enter” for newline. Then enter second parameter referring to the format.

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_target "color:
- 'red'
> - 'green'
ubuntu@ubuntu:~$
```


③ The rest of the parameters should be entered in turn according to the format.

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_target "color:
- 'red'
> - 'green'
> > - 'blue'
> > tag:
> > - 'tag1'
> > - 'tag2'
> > - 'tag3'
```

④ Having finished entering all the parameters, enter " in the last line and press "Enter".

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_target "color:
- 'red'
> - 'green'
> - 'blue'
> tag:
> - 'tag1'
> - 'tag2'
> - 'tag3'
> "
success: True
message: "set_target"
ubuntu@ubuntu:~$
```

2.5 Pause and Quit the Game

You can enter "rosservice call /object_sorting/set_running "data: false"" command to pause the game. Referring to "2.4 Start the game", you can change the targeted color or label after a pause.

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/set_running "data: false"
success: True
message: "set_running"
ubuntu@ubuntu:~$
```

You can enter "rosservice call /object_sorting/exit "{}"" command to quit the game.

```
ubuntu@ubuntu:~$ rosservice call /object_sorting/exit "{}"
success: True
message: "exit"
ubuntu@ubuntu:~$
```

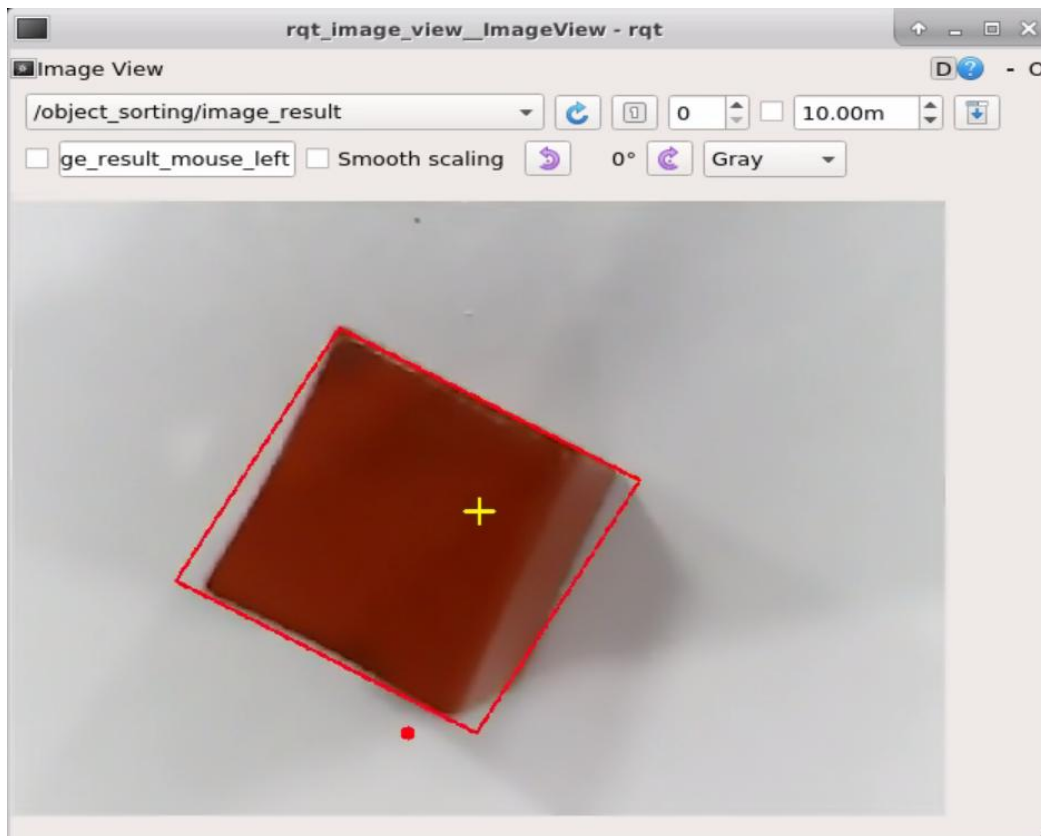
Note: the current game will continue as Raspberry Pi is powered on. To avoid

occupying too much RAM, please quit the current game according to the instruction above before playing other AI vision game.

If you want to turn off the image returned by camera, you can turn back to open rqt terminal and press “Ctrl+C”.

3. Function Realization

Due to the needs of color recognition, please play this game in well-lit room. When the game starts, we should place the red block and the block with Tag1 label on corresponding area of the map. When recognized, the target goods will be framed, which can be seen in the rqt tool. Then the robotic arm will pick the goods with Tag1 label to one side at first and red goods next.

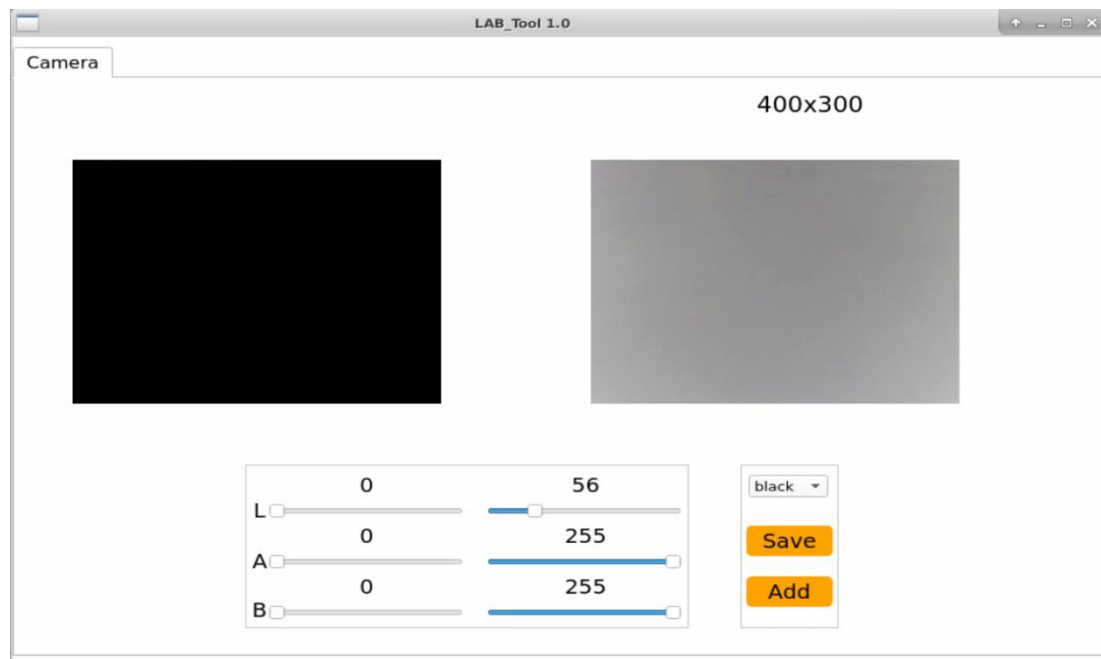
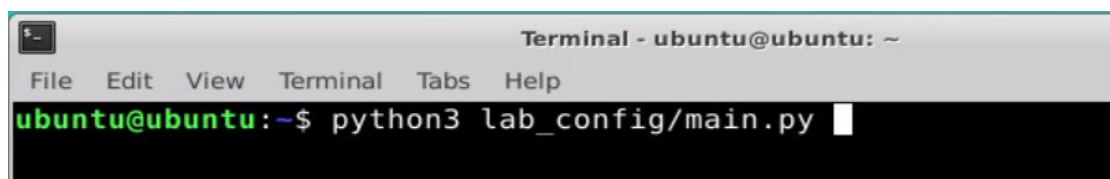


4. Function Extension

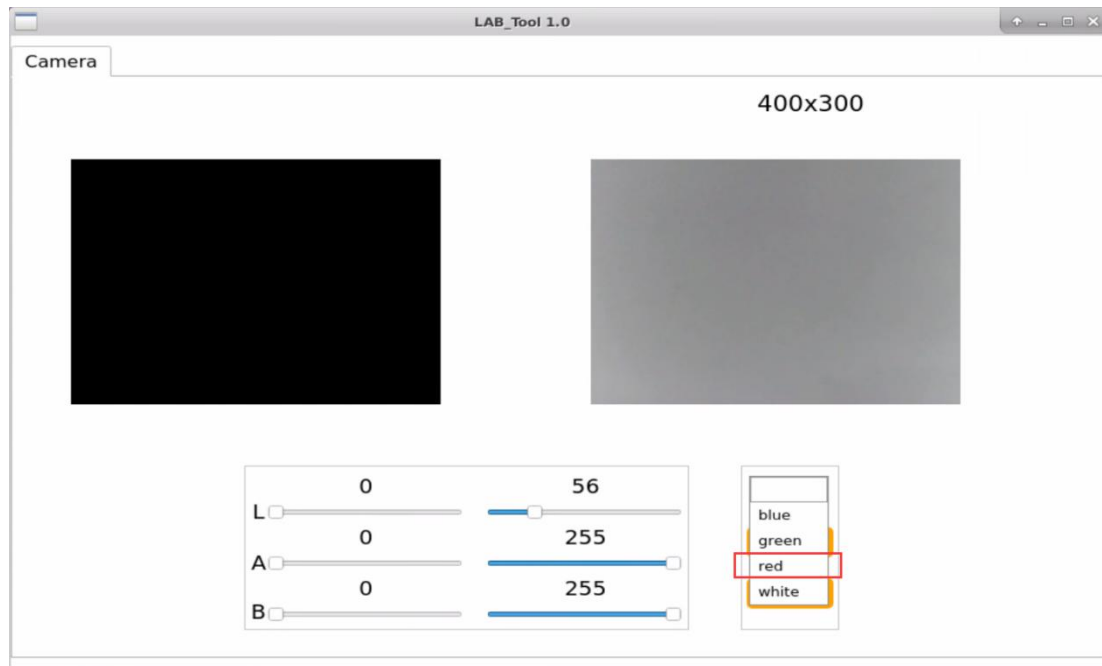
4.1 Add New Recognition Color

There are three built-in recognition colors in the color tracking program, including red, green and blue. Besides these three colors, you can add new recognition colors. Let's take orange for example.

① Please open the terminal interface and enter “python3 lab_config/main.py” command to open color threshold debugging tool. If there is no image returned by camera in the pop-up interface, it means the camera has not been connected successfully. And you should go back to check the camera connection.

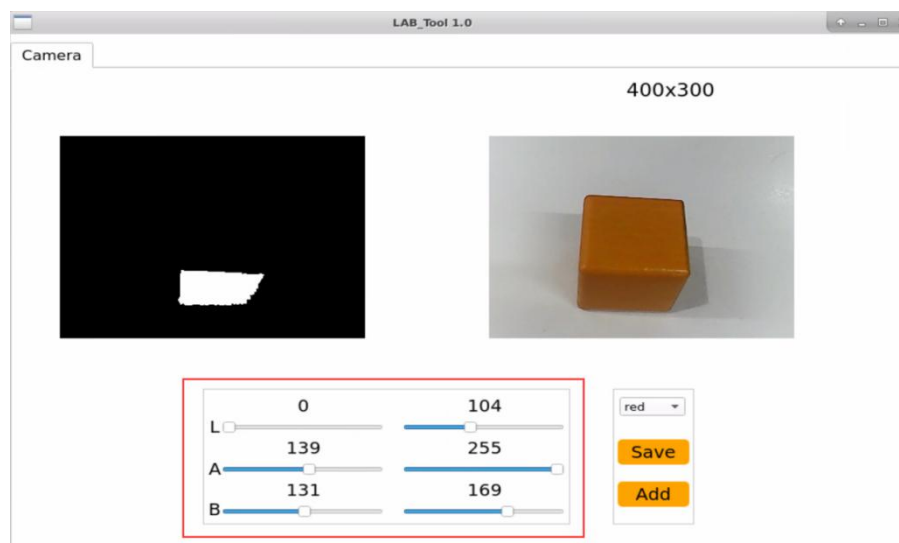


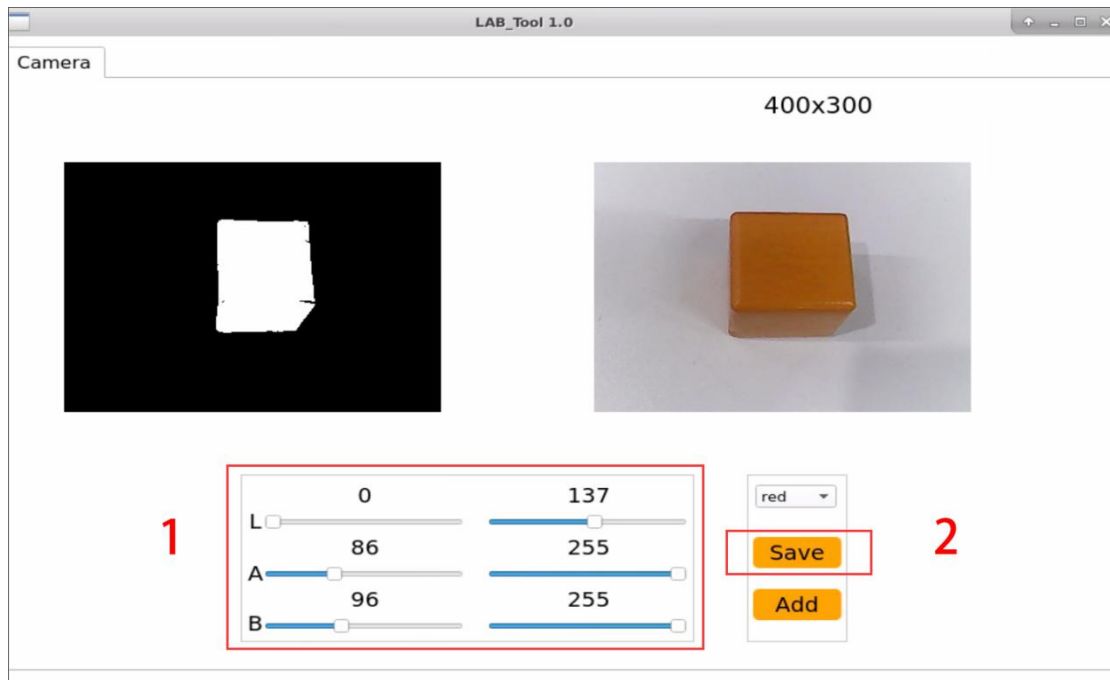
② After connecting successfully, please select “red”. (As shown in the picture)



③ At this time, the image returned by camera is on the right of the interface and the box to recognize color is on the left. Focus the camera on yellow item, then drag the six sliders below until the orange part of the left screen becomes white and area outside of orange become black, and then click " Save" button to keep the modified data.

It is recommended to use screenshot to record the initial value, which is beneficial for you to change the value back. Or you can follow step ③ to set red as the recognition color.





④ Please follow the first three steps in “2. Operation Steps”, from 2.1 to 2.3, to play the goods sorting games.

In the previous steps, we have saved orange as red, therefore we should select the targeted color to “red” before starting the game.

⑤ Please place the orange item in front of the camera. Having recognized, the ArmPi FPV will pick it to color sorting area. If you want to modify to other colors, please operate as the above steps.